



COMSATS University Islamabad, Sahiwal Campus

Spice Safari

By

Muqaddas Nadeem	CIIT/FA21-BCS-009/SWL
Umer Fatima	CIIT/FA21-BCS-030/SWL
Fatima Ghulam Rasool	CIIT/FA21-BCS-038/SWL

Supervisor

Dr. Javed Farzund

Bachelor of Science in Computer Science (2021-2025)

The candidate conforms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University Islamabad, Sahiwal Campus

Spice Safari

**A Project Presented to
COMSATS University, Islamabad
Sahiwal Campus**

**In partial fulfillment of
the requirements for the degree of**

**Bachelor of Science in Computer Science
(2021-2025)**

By

Muqaddas Nadeem

CIIT/FA21-BCS-009/SWL

Umer Fatima

CIIT/FA21-BCS-030/SWL

Fatima Ghulam Rasool

CIIT/FA21-BCS-038/SWL

DECLARATION

We hereby declare that this system, neither whole nor a part, has been copied out from any source. It is further declared that we have developed this project and accompanied the report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Muqaddas Nadeem

Umer Fatima

Fatima Ghulam Rasool

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “Spice Safari” was developed by Muqaddas Nadeem (FA21-BCS-009), Umer Fatima (FA21-BCS-030), and Fatima Ghulam Rasool (FA21-BCS-038) under the supervision of “Dr. Javed Ferzund” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

Dr. Javed Ferzund

Associate Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Iqra Muneer

Assistant Professor

Department of Computer Science, UET Lahore

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Acknowledgment

All praise is to Almighty Allah who bestowed upon us a minute portion of His bondless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Javed Ferzund**”. Without his personal supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty and hard work.

Muqaddas Nadeem

Umer Fatima

Fatima Ghulam Rasool

Executive Summary

Objective

The primary objective of this project is to develop a multi-course food app that provides facility to order easy food for customers by empowering the restaurant to manage its menu and order through a dedicated administrator panel. The app provides an AI-based food recommendations based on the order history and provides nutritional insight to users to help using their food intake. By integrating the Cash on delivery (COD) and Pay Fast payment methods, the app offers safe and convenient transactions options.

1. Scope and Application

In the food industry, the food delivery system that was created for this project is widely used. Through a dedicated administrator panel, the restaurant can now manage the menu, browse their orders, and speed up order handling. Customers can browse, order, and pay with delivery using this menu's user-friendly mobile app. By offering a low-cost and scalable digital platform, the system helps local eateries and startups in urban areas. By examining user input, the integrated nutrition module encourages the consumption of nutritious foods. The system can also be used as a case study for actual e-commerce applications of web-based and mobile technologies by developers and educational institutions.

2. System Design

The development process adheres to agile functioning, guaranteeing consistent advancement, frequent response, and continuing enhancement. The client-server model, which divides the front-end, back-end, and database layers for scalability and mentality, is the foundation of system architecture. A web-based administrator panel for restaurant management, a MySQL database connected via a PHP-based Rest API, and flat-based mobile applications for customers are important components. A responsible and interactive admin interface is created using technologies like HTML, CSS, JavaScript, Bootstrap, and PHP. In order to encourage healthy eating habits, the system also incorporates a nutrition module and payment methods like cash on delivery. Efficient data flow, task management, and future facility integration are made possible by this modular design.

3. Features and Benefits

Several options are available in the food delivery system that are intended to improve efficiency and convenience for both customers and restaurant managers. Clients can personalize orders, view menus, and select safe payment methods like Pay Fast and Cash on Delivery. The integrated nutrition module aids users in tracking and enhancing their eating patterns. Restaurants can monitor orders, manage products, and instantly update order statuses with the admin panel. With centralized management and scalability, the system can accommodate several restaurants. Its modular design enables smooth future upgrades and the integration of new features, while its user-friendly interface guarantees ease of use.

4. System Flow and Quality Features

User registration or login initiates the system workflow, which progresses to menu browsing, item selection, order placing, and payment completion. At the same time, the admin panel receives the order, processes it, and adjusts the delivery status accordingly. The PHP backend, MySQL database, and Flutter-based mobile application all safely exchange information. Because of its timeliness and dependability, the system operates smoothly even when there is a significant user load. Its modular architecture allows for future improvements and the integration of more eateries. Its emphasis on performance, usability, and maintainability ensures user happiness and reliability over time.

Abbreviations

SRS	Software Require Specification
SDD	Software Design Description
GUI	Graphical User Interface
IDEs	Integrated development environments
PHP	Hypertext Preprocessor (Programming Language)

Table of Contents

1	Introduction.....	1
1.1	Purpose.....	1
1.2	Features of Food Delivery App.....	1
1.3	Significance of Project	2
2	Problem-Solving	4
2.1	Problem Identification.....	4
2.1.1	Inefficient Ordering Process	4
2.1.2	Lack of Personal	4
2.1.3	Limited Restaurant Accessibility.....	4
2.1.4	Deferred Customer Support	4
2.1.5	Lack of Nutritional Awareness	5
2.1.6	Accessibility Challenges	5
2.1.7	Manual Restaurant Management	5
2.1.8	Problem-Solving Approach.....	5
2.2	Benefits of the Solution.....	6
3	Requirement Analysis	8
3.1	Functional Requirements.....	8
3.1.1	Customer Module.....	8
3.1.2	Admin Module	9
3.1.3	Restaurant Module	10
3.2	Non-Functional Requirements	10
3.2.1	Performance	10
3.2.2	Usability	10
3.2.3	Compatibility	11
3.2.4	Data Integrity	11
3.2.5	Scalability	11
3.3	System Constraints	11
3.4	Use Case Scenario	12
3.5	Detailed Use Case	15
3.5.1	Login.....	15
3.5.2	View Menu.....	16

3.5.3	Place Order.....	16
3.5.4	Checkout	17
3.5.5	Cancel Order	18
3.5.6	Add/ Edit/Delete Menu	19
3.5.7	Add New Feature	20
3.5.8	Logout.....	21
4	Design and Architecture.....	23
4.1	System Architecture	23
4.2	Process Flow	24
4.2.1	App Initiation and User Authentication.....	24
4.2.2	Restaurant and Menu Browsing.....	24
4.2.3	Cart Management.....	25
4.2.4	Checkout & Payment	25
4.2.5	Admin Order Management	25
4.2.6	Completion of Order and Feedback.....	25
4.2.7	Standby and Subsequent	25
4.3	ERD Diagram.....	27
4.4	Sequence Diagram.....	28
5	Implementation of Spice Safari Food Delivery App.....	32
5.1	Overview	32
5.2	Tools and Technologies	32
5.3	Modules and Their Implementation	32
5.3.1	User Registration and Authentication Module	33
5.3.2	Restaurant Management Module (Admin Portal).....	33
5.3.3	Menu Display and Ordering Module (Customer App).....	33
5.3.4	Payment and Checkout Module.....	33
5.3.5	Admin Dashboard and Analytics Module.....	33
5.3.6	Review and Rating Module	33
5.3.7	Order Tracking Module	34
5.4	Workflow	34
5.5	Testing and Deployment	34
5.6	Challenges and Solutions	35
5.7	Home Screen	36
5.8	Categories and Subcategories.....	37

5.9	Cart.....	38
5.10	Checkout.....	39
5.11	Restaurant Admin.....	40
5.12	Add Categories.....	40
6	Testing and Evaluation	43
6.1	Manual Testing.....	43
6.2	System Testing	43
6.3	Unit Testing.....	43
6.4	Functional Test Cases.....	44
6.5	Authentication and Session Management Testing	45
6.6	Integration Testing	45
6.7	User Acceptance Testing (UAT).....	46
6.8	Summary	46
7	Conclusion	48
8	Future Work	48
8.1	Automated Rider Assignment	48
8.2	Loyalty and Reward System	48
8.3	Performance Analytics Dashboard.....	48
8.4	Enhanced Security and Data Privacy	49
9	References.....	50

Table of Figures

Figure 3.1 Use Case Diagram	14
Figure 4.1 System Architecture of Food app	24
Figure 4.2 Process Flow Diagram.....	26
Figure 4.3 Entity Relationship Diagram	27
Figure 4.4 Sequence Diagram of Food app.....	28
Figure 4.5 Sequence Diagram of Super Admin	29
Figure 4.6 Sequence Diagram of Restaurant.....	30
Figure 5.1 Main Screen.....	36
Figure 5.2 Subcategory	37
Figure 5.3 Cart	38
Figure 5.4 Checkout.....	39
Figure 5.5 Restaurant Admin.....	40
Figure 5.6 Add Categories Super Admin	41

List of Tables

Table 3.1 Login.....	15
Table 3.2 View Menu.....	16
Table 3.3 Place Order.....	17
Table 3.4 Checkout.....	18
Table 3.5 Cancel Order.....	18
Table 3.6 Add/Edit/Delete.....	19
Table 3.7 Add New Feature.....	20
Table 3.8 Logout.....	21
Table 6.1: Customer Order Placement Testing.....	44
Table 6.2: Admin Login and Dashboard Access.....	45
Table 6.3 Integration Testing.....	45

Chapter No. 1

Introduction

1 Introduction

Our app enhances the modern dining experience by providing an easy, quick, and convenient way of ordering food. Rather than calling restaurants or standing for extended lines, customers can quickly browse through menus, pick their favorite foods, and order directly from their mobile phones. The app accommodates several restaurants so customers can pick from a vast choice of cuisines from a single destination. Secure payment options such as Cash on Delivery (COD) and Pay fast make the process safe as well as dependable. The nutrition reviewing module is an unusual feature in our app which enables users to track their eating habits by entering basic lifestyle information. This supports healthier eating habits and increases consumer trust. Social aspects like recommendation, rating, and review enable the user to make food choices with confidence. The special offering of our app is the nutrition checking feature, through which users can track their dietary patterns by providing simple information regarding their lifestyle. This helps promote healthy food choices as well as user trust building. The app uses AI to suggest dishes to customers based on their ordering history, providing personalized recommendations based on their preferences. Clean, intuitive interface helps with smooth navigation, even for non-technical customers. Menu, price, and order can be managed by restaurants through the admin interface, promoting local food businesses.

1.1 Purpose

The role of our food ordering application is to improve the dining experience by offering a smooth, convenient, and secure way for customers to view meals, place food orders, and Users can easily pay for their meals directly from their phones, making the checkout process fast and convenient. The application connects restaurants with customers by providing simple ordering, and direct communication. The application helps support neighborhood food businesses while giving customers a seamless, satisfying food ordering experience. The app is designed to be easy for users of all ages with a smartphone, making ordering food quicker, smarter, and more enjoyable.

1.2 Features of Food Delivery App

Our food delivery application offers a range of functional as well as user-centric features including:

1. **Multi-restaurant support:** customers can search through menus and place orders from multiple restaurants within one app.

2. **User Account System:** Customers can register, log in, and control their profile and purchase history.
3. **Secure Payments:** Cash on Delivery option as well as Pay fast provide for secure transactions.
4. **Search & Filter Menu:** Locate dishes by keyword or through category, price, etc.
5. **Ratings and Reviews:** Check what other customers have said before you order.
6. **Nutrition Module:** Lets users track their eating habits according to their food consumption.
7. **AI-Based Food Suggestions:** Recommends food based on the user's preferences, past orders, and current trends for higher personalization and satisfaction
8. **Categories:** Burgers, Beverages, Desi Food, Pizzas, Rice,
9. **Favorite/Recent Order Items:** User is able to reorder easily based on items he liked or order recently.
10. **Admin Dashboard with Analytics:** Graphs/Statistics illustrating total orders, revenue.
11. **Location Services Integration:** Autodetect the user's location for restaurants nearby or express delivery
12. **Order History:** Previous orders are viewable by users.
13. **Chat Support:** Chat with support for support issues or questions.
14. **Admin Panel Access:** The owners of restaurants can manage orders, prices, and menus.
15. **User-Friendly UI:** Simple, intuitive interface for customers as well as for admins.

1.3 Significance of Project

Our project goes beyond just building a food ordering app, it supports local businesses, promotes digital growth in the food industry, and makes the customer experience better. By offering easy access to food categories, smart AI recommendations, reorder history, and quick customer support, the app creates a complete, modern dining experience. It helps customers save time, choose better meals, while giving restaurants better tools to run their businesses smoothly. In today's fast-moving world, our app makes food ordering simpler, smarter, and more user-friendly.

Chapter No. 2

Problem Definition

2 Problem-Solving

The digital age has transformed how people use food delivery services, but challenges like confusing menus, slow customer support, and difficulty finding the right food still exist. Our food delivery app, powered by artificial intelligence, solves these problems by offering smart features like organized food categories, AI-driven recommendations, and easy access to order history for quick reordering. The app also provides strong customer support, making the entire experience faster, easier, and more personalized for users.

2.1 Problem Identification

Most food delivery apps lack multi-restaurant support, AI recommendations, order tracking, and admin control, leading to limited user experience and inefficient management.

2.1.1 Inefficient Ordering Process

Traditional ordering processes include calling restaurants, as well as utilizing simple web sites with minimal features. These processes will often cause miscommunication, delays, and inconvenience. Our application simplifies the ordering process by providing easy-to-use navigation as well as instant confirmation.

2.1.2 Lack of Personal

Most food applications have default recommendations that do not match users' choices. Customers spend time viewing items that are not relevant. The AI recommendation system learns what users like and suggests food based on their past orders.

2.1.3 Limited Restaurant Accessibility

Most food delivery systems have only one supported restaurant. This forces users to deal with limited restaurant choices and download multiple apps. Our solution is able to support several restaurants within one framework, allowing users a centralized, streamlined experience.

2.1.4 Deferred Customer Support

Customers often encounter issues with their orders, and unresponsive support can lead to frustration. We've integrated live chat support directly within the app to address this. With just a click of a button, customers are instantly connected to our dedicated support team, ensuring quick and efficient resolution of any problems. Seamless integration ensures smooth, timely customer assistance.

2.1.5 Lack of Nutritional Awareness

Excessive fast-food consumption is unhealthy, and customers do not often have the tools for controlling their diet. Our nutrition module promotes healthy eating by tracking user habits and providing them with food consumption feedback.

2.1.6 Accessibility Challenges

Limited mobility or technical expertise could be difficult for users who have difficulty navigating complex app interfaces. Our app offers a simple, intuitive UI that improves usability for users of all experience levels.

2.1.7 Manual Restaurant Management

Menu, price, and order handling are challenges for restaurants with no centralized system. Our admin panel helps restaurant owners manage their digital activities efficiently in real time.

2.1.8 Problem-Solving Approach

Our food ordering system operates on a user-centric model that combines the latest technology, artificial intelligence, and automation to address actual issues of users as well as restaurateurs.

2.1.8.1 Key Problem-Solving Strategies

Implementing a multi-restaurant system with AI-powered recommendations, real-time order tracking, nutritional info, and an admin dashboard to enhance both user and admin experiences.

1. Smart Food Ordering System

Enables users to browse categorized menus, view dish images, and place orders quickly with a simple and intuitive interface, reducing confusion and effort during the ordering process.

2. Personalized User Experience

Offers customized recommendations and saves order history for quick reordering, ensuring a smooth and tailored experience based on user preferences and behavior.

3. Nutrition Tracking

Assists users in becoming health-conscious by examining food habits and eating patterns.

4. Chat-Based Support System

Improves customer service through built-in real-time support within the app.

5. Administrative Management Tools

Facilitates restaurant partners managing their price, and menu with ease through a secure admin panel.

2.2 Benefits of the Solution

1. **Enhanced Efficiency:** Saves time spent on ordering food by providing smart search, filtering, and reordering capabilities.
2. **Improved User Experience:** Clean interface, live feedback, and intelligent features make for a seamless, pleasant experience.
3. **Health and Wellness Awareness:** The nutrition module encourages users to adopt healthy eating habits.
4. **Accessibility for All:** User-friendly design makes for an easy-to-use facility for anyone regardless of age, ability, or disability.
5. **Increased Restaurant Visibility:** Multi-restaurant support extends the reach of customers for local food businesses.
6. **Intelligent decision-making:** AI suggestions and insights guide customers into making wiser decisions.
7. **Scalable & Future:** The system can easily support new functionalities such as voice ordering, enhanced analytics, or extra payment methods.

Chapter No. 3

Requirements Analysis

3 Requirement Analysis

Requirement analysis is an important step in developing the food ordering app. This section outlines the necessary functional and non-functional requirements to ensure the system works well, is efficient, and can grow in the future. The goal is to understand what the system needs to do, the tools it requires, and any limitations to consider during development.

3.1 Functional Requirements

Functional requirements specify the primary functions the system should accomplish to satisfy its purpose. These are the following modules that will be included in this system.

1. Customers Module
2. Admin Module
3. Restaurant Module

3.1.1 Customer Module

The customer module in the functional requirements of a food ordering app typically includes a range of features to ensure a seamless and engaging user experience. Here are the key elements:

1. **User Registration and Login:** Users register and log in using a custom PHP-based authentication system. Secure validation for strong passwords and proper email format. Authenticated users gain access to features like food browsing, cart, and order history.
2. **Food Item Browsing:** Browse menus across multiple restaurants. Categorized menu (Burgers, Biryani, etc.) Items display name, image, price, and ratings.
3. **Cart Management:** Add, remove, and update items in cart. Supports price variants (e.g., half/full plate). Singleton pattern in Flutter ensures state persistence.
4. **Order Placement:** Enter delivery address and contact info. Choose payment method (COD or online). Confirm and save order to MySQL database.
5. **Multi-Restaurant Support:** Users browse and order from multiple restaurants. Orders routed to respective restaurants.
6. **Nutrition:** Provides basic advice based on fast food and water intake. Simple logic model for feedback (e.g., drink more water).
7. **User Profile Management:** View/edit name, phone, and other details. Access and reorder from order history.

8. **Rating and Review System:** Rate and review food and restaurants post-delivery Contributes to service quality feedback.
9. **List of Favorites:** Mark favorite items/restaurants for quick access.
10. **Contact/Support System:** Submit queries or issues via form/chat Admins receive/support via WhatsApp.

3.1.2 Admin Module

The admin module in the functional requirements of a food ordering app typically includes a range of features to ensure a seamless and engaging user experience. Here are the key elements: An overview of common functional needs for the admin module is provided below:

1. **Admin Login and Access:** Secure login via standalone web interface (PHP authentication) Access restricted based on admin credentials.
2. **User Management and Registration** Add, edit, remove, and view user accounts. View user profiles and order history Manage user roles and access permissions.
3. **Restaurant Management:** Add, edit, or remove restaurant entries Modify restaurant details (name, address, phone, status) Assign restaurant_id to link with respective admin.
4. **Menu Configuration and Management:** Add/edit/remove menu items per restaurant Organize menus into categories and subcategories Manage item prices, descriptions, and images.
5. **Order Tracking and Management:** View and manage all incoming orders Track delivery status and payment confirmation Assign or update order statuses (pending, confirmed, delivered).
6. **Payment Management:** The system manages PayFast and Cash on Delivery payment methods, securely integrates gateway settings, ensures correct method selection during order placement and provides a user-friendly interface for managing payment related issues.
7. **Review and Feedback Moderation:** Monitor user reviews for food, restaurants, and delivery staff Remove inappropriate reviews or flag issues Respond to customer feedback to maintain service quality.
8. **Customer Support Management:** A contact/support form or chat facility where users can report issues or ask questions, and admins can view inquiries received on WhatsApp.

3.1.3 Restaurant Module

Restaurant module facilitates order management, smooth operations, accurate handling, and real-time updates for businesses. An overview of common functional needs for the **restaurant module** is provided below:

1. **Multi-Restaurant Support:** Each restaurant has its own admin and unique data Admins are associated with a restaurant_id for secure and filtered access Customers can browse and order from multiple restaurants independently.
2. **Menu Creation and Management:** Add, edit, or delete menu items linked to that restaurant Organize menu into logical sections (e.g., desi food, drinks, desserts) Define prices, write descriptions, and upload images for each item.
3. **Order Management:** View and manage all incoming orders specific to the restaurant. Update order status (Pending, Delivered and cancelled) and access customer contact information for each order.

3.2 Non-Functional Requirements

Non-functional specifications outline the characteristics of the system in terms of how it should operate, rather than what it does. These specifications ensure that the application is efficient, secure, scalable, and user-friendly.

3.2.1 Performance

The application should respond to all the most important user actions (e.g., browsing menu, cart addition, orders placement) under 2 seconds. Backend (PHP + MySQL) should be able to support simultaneous multiple users, but not with any delay or crashes. Queries to the MySQL database are optimized with indexes and caching to minimize latency. Images and menus are efficiently loaded using lazy loading methods to enhance the speed.

3.2.2 Usability

The user interface should be neat, modern, and user-friendly, accessible to individuals of all ages, including non-technical individuals. Buttons, icons, and input fields should be properly sized and readily tappable on any Android screen size. Consistency of colors, fonts, and design elements is applied throughout the app to enhance the user experience. The admin panel has been set up with straightforward navigation, convenient access to crucial actions, and live updates (e.g., orders incoming).

3.2.3 Compatibility

The application is developed with Flutter and has to be compatible with all Android devices using Android version 8.0 (Oreo) or higher. The web admin panel is responsive and should work well with all the leading browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge. The UI adapts to screen size so that the admin panel can be accessed on desktops, tablets, and mobile browsers.

3.2.4 Data Integrity

The user accounts and the personal information are managed using a custom-made PHP & MySQL backend. All the information is sanitized to block known attacks SQL Injection. Delivery addresses, preferences, and order histories are kept securely, not to be revealed without the user's approval. Role-Based Access Control (RBAC) guarantees. Each user has access to only his/her data. Admins can only access management tools of their restaurant.

3.2.5 Scalability

Scalability is a crucial factor for any online meal ordering system. The system should be able to handle increased traffic and orders without sacrificing performance or dependability. The system should also be adaptable to one restaurant and locations, enabling for future growth and expansion.

3.3 System Constraints

The application is compatible across multiple platforms, ensuring smooth functionality on both Android devices and web browsers for admins.

1. Technical Limits

The system runs on local hosting (XAMPP), which prohibits live spirit scalability. No integration with cloud services (e.g., firebase, AWS). Only Android devices are supported (no iOS version is available). Push notifications have not been implemented in the present.

2. Lack of Time and Budget

The system was developed within a limited academic program. Due to budget limitations: No use of paid API or commercial backend tools. Minimum UI/UX test in devices. Advanced facilities like AI-based recommendations or voice search are postponed.

3. Shortage of Resources

This app was developed by a student team of three, with limited industry experience. All development, testing and UI designs were made at home with basic equipment.

3.4 Use Case Scenario

The system enables customers to browse menus and order, while admins manage dishes and track orders via dashboard (see Figure 3.1).

1. Login

- The administrator enters its credentials (email/phone number and password) and presses the login button.
- The system validates the credentials recorded and successfully logs into the system.
- The system confirms: "You have successfully logged in."

2. Registration

- The customer navigates on the registration page and records their information (email, user name, password, first and last name).
- The system verifies information and creates a new account for the customer.
- The system confirms: "Registration successful, now you can log in."

3. Browse Menu

- The customer logs in to the system and requests to see the menu.
- The system displays the menu with available items and allows the customer to apply the filter based on the price and rating.
- The customer sees the details of each item (name, details, price and rating).
- The system confirms: "Menu displayed, now you can browse items."

4. Place Order

- The customer browses the menu, selects some items, and proceeds to checkout.
- Calculates total and requests payment details.
- The customer records payment details and confirms the order.
- The system processes the payment and confirms the order.

5. Checkout

- The customer reviews the item in his car and adjusts the volume as required.
- The customer confirms the distribution address and proceeds to pay.
- The system displays available discounts and confirms the details. Customer enters payment information and income.
- The system confirms: "Checkout successful, processed payment."

6. Cancel Orders

- Customer navigates in the list of orders and selects the order that wish to cancel.
- The system prompts confirmation, and the customer confirms the cancellation.
- The system changes the order status to "Canceled" and informs the customer.
- The system confirms: "Order canceled successfully."

7. Feedback

- After receiving their order, the customer is motivated to quit the response to their experience.
- The customer offers comments about the quality of food and service.
- The system stores and confirms: "Thanks to your response."

8. Add/Edit/Delete Menu

- The owner of the restaurant logs into the system and decides to add a new item to the menu.
- The system menu presents the option of adding, editing or removing the item.
- The owner of the restaurant enters the item description (name, description, price) and saves it.
- The system updates the menu and confirms: "The menu was successfully updated."

9. Take Orders

- Restaurant employees receive an order request from a customer.
- The employee assists the customer in selecting the menu item and records the order.
- The system confirms the order details with the customer and verify them.
- The system confirms: "The order was successfully taken."

10. Add New Features (Administrator)

- The administrator logs into the system and reaches the analytics dashboard.
- The administrator selects to see the order trend and customer response.
- The system confirms: "Analytics data was successfully displayed."

11. Logout

- The administrator navigates his profile and clicks on the logout button.
- The system logs to the administrator from the system and redeals them to the login screen.
- The system confirms: "You have successfully logged out."

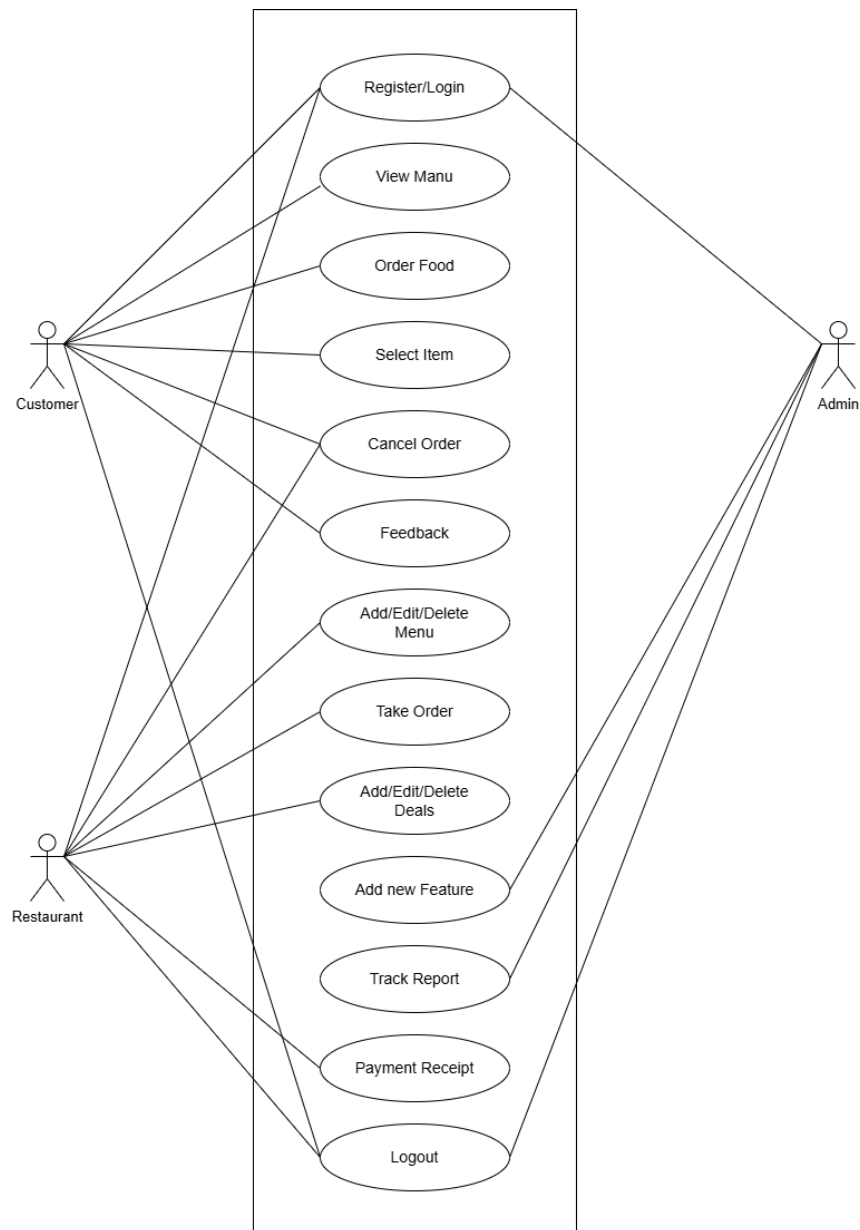


Figure 3.1 Use Case Diagram

3.5 Detailed Use Case

The Table below indicates a comprehensive use case template filled in with an example drawn from the Food delivery app.

3.5.1 Login

The login system securely authenticates users, granting access to customer or admin features based on their credentials (see Table 3.1).

Table 3.1 Login

Use case id	01
Use case name	Get Registration / Login
Actors	• Customer • Admin • Restaurant
Description	A user of the system creates an account or logs in.
Trigger	User selects the login or registration option from the system.
Pre-condition	The user does not have an account or is not currently logged in.
Postcondition	The user's details are securely stored in the database, and access is granted accordingly.
Normal flow	User accesses the login/registration page. System prompts for email and password or new registration. User selects the registration option. System displays a sign-up form (username/email, first name, last name, password). User enters details. System verifies and creates the account or logs the user in.
Alternative flow	Cancel Registration: User selects cancel. System returns to the homepage and clears input. Invalid Information: System detects incorrect input. Displays error message User re-enters correct information.
Exceptions	"User already exists" if the email is registered. "Insufficient Information" if mandatory fields are missing.

3.5.2 View Menu

Customers can view the available menu items categorized by type, along with prices and details (see Table 3.2).

Table 3.2 View Menu

Use case id	02
Use case name	View Menu
Actors	Customer
Description	Allows the customer to view the menu items available in the system.
Pre-condition	The system is accessible and the customer is logged in.
Postcondition	The customer has viewed the menu items.
Normal flow	The use case starts when a customer wants to view the menu. The system presents the customer with the option to browse the menu. The customer can apply filters to the menu items based on price and ratings. The customer can view the details of each menu item, such as name, description, price, and ratings.
Alternative flow	The user did not register him/herself and did not view the available menu details.
Exceptions	The error occurs in the application.
Business rules	Active internet access is available.

3.5.3 Place Order

Customers can place orders by selecting desired items, specifying quantities, and confirming their choices through a streamlined interface (see Table 3.3).

Table 3.3 Place Order

Use case id	03
Use case name	Place Order
Actors	Customer
Description	Allows the customer to place a food order.
Pre-condition	The system is accessible and the customer is logged in.
Postcondition	The customer has successfully placed the food order.
Normal flow	The use case starts when a customer wants to place a food order. The system presents the customer with the option to browse the menu and select items. The customer browses the menu and selects the desired items for their order. The customer can customize their order with extras like toppings or special instructions. The customer confirms the order and proceeds to checkout. The system calculates the total order amount including taxes and delivery charges. The customer selects the preferred payment method and provides the necessary details. The system processes the payment securely and confirms the order.
Alternative flow	The customer decides to cancel the order before finalizing the payment. In this case, the system cancels the order and notifies the customer.
Exceptions	The application shows an error or the server is down during the order placement process.
Business rules	Active internet access is required to place the order. The customer must be registered and logged in to place the order. The customer must provide valid payment information to complete the order.

3.5.4 Checkout

During checkout, customers can review their order, select a payment method, and confirm delivery details to complete the transaction (see Table 3.4).

Table 3.4 Checkout

Use case id	04
Use case name	Checkout
Actors	Customer
Description	Allows the customer to review and finalize their food order before payment.
Pre-condition	The system is accessible and the customer has items in their cart.
Postcondition	The customer has reviewed their order and proceeded to payment.
Normal flow	The customer reviews the items in their cart, making any necessary adjustments such as quantity or removing items. The customer confirms their delivery address and selects the preferred delivery time if applicable. The customer reviews any applicable discounts or promotional offers applied to their order. The customer proceeds to payment by selecting the preferred payment method and providing the necessary payment details.
Alternative flow	If the customer decides to cancel the order during the checkout process, they can return to browsing or cancel the order entirely.
Exceptions	The application encounters an error or the server is down during the checkout process.
Business rules	Active internet access is required to proceed with checkout. The customer must be registered and logged in to complete the checkout process. The customer must provide valid payment information to proceed with payment.

3.5.5 Cancel Order

Customers have the option to cancel their order within a limited time frame before processing begins (see Table 3.5).

Table 3.5 Cancel Order

Use case id	05
Use case name	Cancel Order

Actors	• Customer • Restaurant
Description	Allows the customer or the restaurant to cancel a food order.
Pre-condition	The system is accessible and an order exists to be canceled.
Postcondition	The order is successfully canceled.
Normal flow	The user selects the order they want to cancel from the list of orders. The system confirms the cancellation request with the user. The user confirms the cancellation. The system updates the order status to "canceled" and notifies both parties involved.
Alternative flow	If the user changes their mind before confirming the cancellation, they can choose to keep the order.
Exceptions	The application shows an error or the server is down during the cancellation process.
Business rules	Active internet access is required to cancel the order. The user (customer or restaurant) must be authorized and logged in to cancel the order.

3.5.6 Add/ Edit/Delete Menu

Restaurant owner can easily add, edit, or delete menu items, including updating names, prices, descriptions, and images to maintain an accurate and appealing menu (see Table 3.6).

Table 3.6 Add/Edit/Delete

Use case id	06
Use case name	Add/Edit/Delete Menu
Actors	Restaurant Owner
Description	Allows the restaurant owner to manage the menu by adding, editing, or deleting menu items.
Pre-condition	The system is accessible and the admin is logged in.
Postcondition	The menu is successfully updated with the changes made.
Normal flow	The use case starts when admin wants to manage the menu. The system presents the admin with options to add, edit, or delete menu items. If adding a new menu item, the restaurant owner provides the details such as name, description, price, and category.

	If editing an existing menu item, the admin selects the item to edit and modifies the necessary details. If deleting a menu item, the admins select the item to delete from the menu. The system updates the menu accordingly based on the restaurant owner's actions.
Alternative flow	If the admin decides to cancel the action before saving the changes, they can choose to discard the changes.
Exceptions	The application shows an error or the server is down during the menu management process.
Business rules	Active internet access is required to manage the menu. The admin must be authorized and logged in to perform menu management tasks. Menu items should adhere to any regulations or guidelines set by the governing authorities.

3.5.7 Add New Feature

A new feature allows admins to highlight special offers or featured dishes on the homepage to attract customer attention (see Table 3.7).

Table 3.7 Add New Feature

Use case id	07
Use case name	Add New Features
Actors	Admin
Description	Administrators view and analyze various data related to orders, customer interactions, and restaurant performance.
Pre-condition	The system is accessible and the admin is logged in.
Postcondition	The administrator has accessed and analyzed the desired data through the analytics dashboard.
Normal flow	The functionality starts when an administrator wants to access the analytics dashboard. The administrator navigates to the "Analytics" section in the app. The administrator selects the type of data to analyze, such as order trends, customer feedback, or sales performance. The system retrieves and displays the relevant data in the form of charts, graphs, or tables. The administrator analyzes the data to identify patterns, trends, and areas for improvement.

Alternative flow	If the administrator wants to customize the data view or apply filters, they can do so using the options provided in the dashboard.
Exceptions	The application shows an error or the server is down during the data retrieval process.
Business rules	Active internet access is required to access the analytics dashboard. Administrators must be authorized and logged in to access the analytics dashboard. The analytics dashboard should provide valuable insights to support decision-making and business strategies.

3.5.8 Logout

Users can securely log out of their accounts to ensure privacy and prevent unauthorized access (see Table 3.8).

Table 3.8 Logout

Use case id	08
Use case name	Logout
Actors	• Admin • Customer
Description	Allows users (customers, administrators, etc.) to log out from the app.
Pre-condition	The system is accessible and the user is logged in.
Postcondition	The user is logged out from the app and redirected to the login screen.
Normal flow	The user navigates to their profile or settings section. They locate the "Logout" button or option. The user confirms the logout action. The system logs out the user and returns to the login or home screen.
Alternative flow	If the user is in the middle of a task, such as placing an order or editing their profile, and attempts to logout, a confirmation dialog appears.
Exceptions	If the logout action fails due to technical issues, an error message is displayed, and the user remains logged in.
Business rules	Active internet access is required for logging out. Users must be authorized and logged in to access the app. Any unsaved changes or pending transactions should be handled appropriately upon logout.

Chapter No. 4

Design and Architecture

4 Design and Architecture

Design software development plays an important role in the life cycle, reduces the difference between need analysis and real implementation. While the architecture provides a high-level observation of the entire system, design phase components, modules, data flows and user dive into interactions that form the system. This chapter underlines the system architecture, process flow and major interaction sequences of the food distribution app.

4.1 System Architecture

The architecture of the food delivery app is made up of many interconnected modules designed to make online food ordering and delivery services facilitated. Each component plays a crucial role in ensuring seamless communication, efficient order processing, and real-time updates. The system is divided into three primary components: customer app, admin panel, and backend server (API + database) (see Figure 4.1).

1. Customer App (Flutter - Dart)

Customer-supporting app allows users to browse restaurants, detect menu, add items to carts, order and track delivery. Cross-platform is designed using pulsing for support. It connects with backend using restful APIs for real -time data operation.

2. Administrator Panel (Web-Based)

HTML, CSS, JavaScript, bootstrap and Ajax are made using. The restaurant allows owners/praise to manage their menu, view, update items availability and analyze the performance. Each restaurant has a separate administrator panel (multi-restaurant system).

3. Backend Server (using PHP + MySQL)

Centralized arguments for API requests, user authentication and data operations. MySQL database stores information on users, restaurants, orders, menu items and payment. The PHP handles the interaction between the server-side scripting and the app, the admin panel and the database.

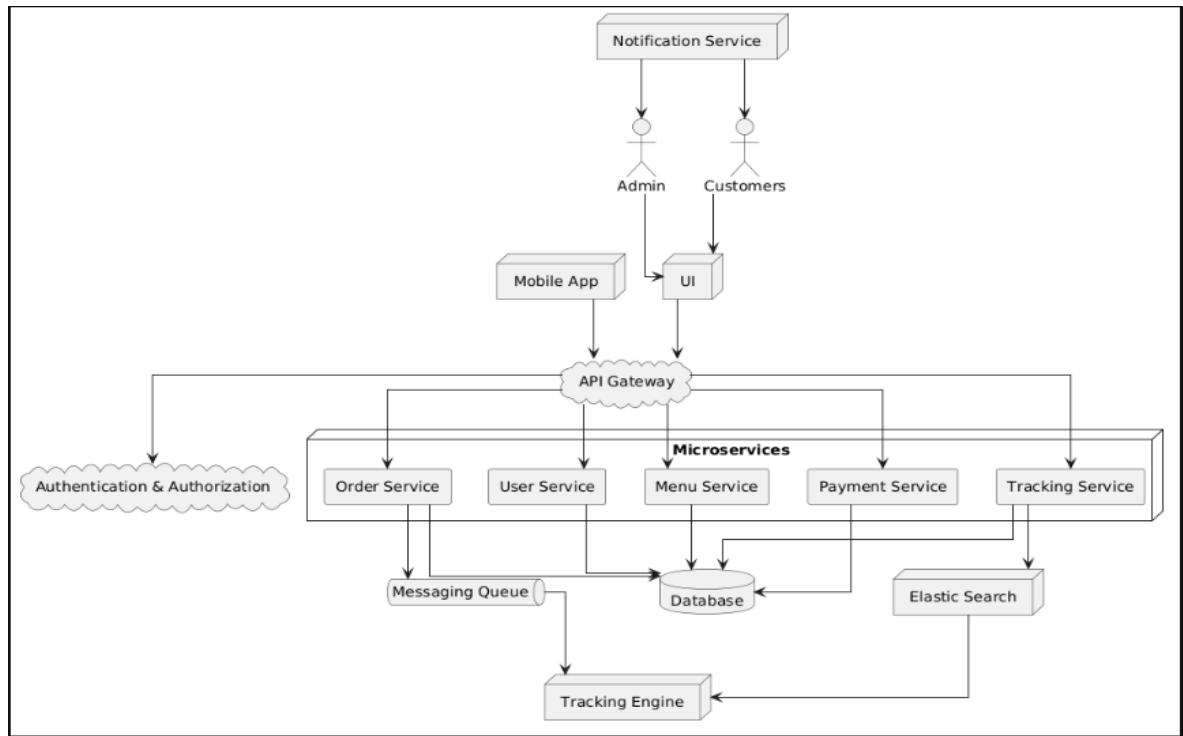


Figure 4.1 System Architecture of Food app

4.2 Process Flow

Process flow defines the step-by-step progression from user sign-in to completion of the order. This step-by-step process guarantees seamless interaction and optimal performance between the app, backend, and admin interface (see Figure 4.2).

4.2.1 App Initiation and User Authentication

The application starts with a splash screen and requires secure user authentication, subsequently guiding users to specific dashboards by role. Users can either log in or sign up through MYSQL Authentication. Upon successful login, user profiles are loaded, leading to the home screen.

4.2.2 Restaurant and Menu Browsing

Users can explore restaurants and view detailed menu items, which include dish descriptions, prices, images, and quantities. The app displays restaurants categorized by food types (e.g., Burgers, Drinks, Pizza).

4.2.3 Cart Management

Users manage their cart by adding, updating, or removing items, with real-time price calculations and quantity adjustments.

4.2.4 Checkout & Payment

The checkout phase allows customers to review their order, select delivery details, and complete payments via Cash on Delivery (COD) or PayFast integration, with order confirmations sent through a RESTful API.

4.2.5 Admin Order Management

Admins manage customer orders with functionalities to view and update order status, ensuring real-time synchronization through API calls.

4.2.6 Completion of Order and Feedback

Post-delivery, customers provide valuable feedback and ratings, enhancing service quality.

4.2.7 Standby and Subsequent

The system manages standby orders and resets the cart after an order is completed, returning the app to the home screen for new orders.

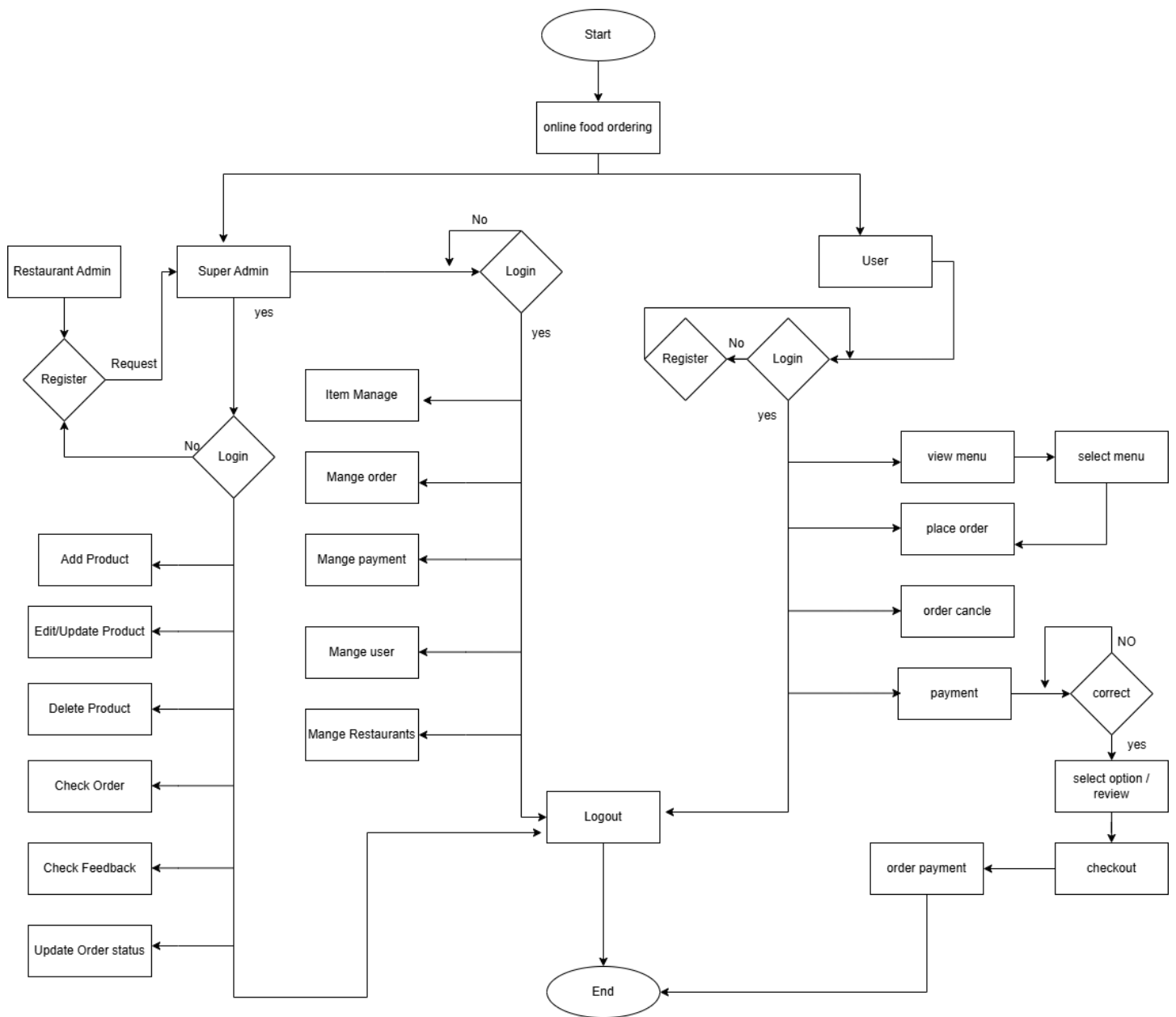


Figure 4.2 Process Flow Diagram

4.3 ERD Diagram

An **ERD** is a visual representation of the data structure within a system. It shows **entities** (like users, products, orders), their **attributes** and the **relationships** between them. (see Figure 4.3).

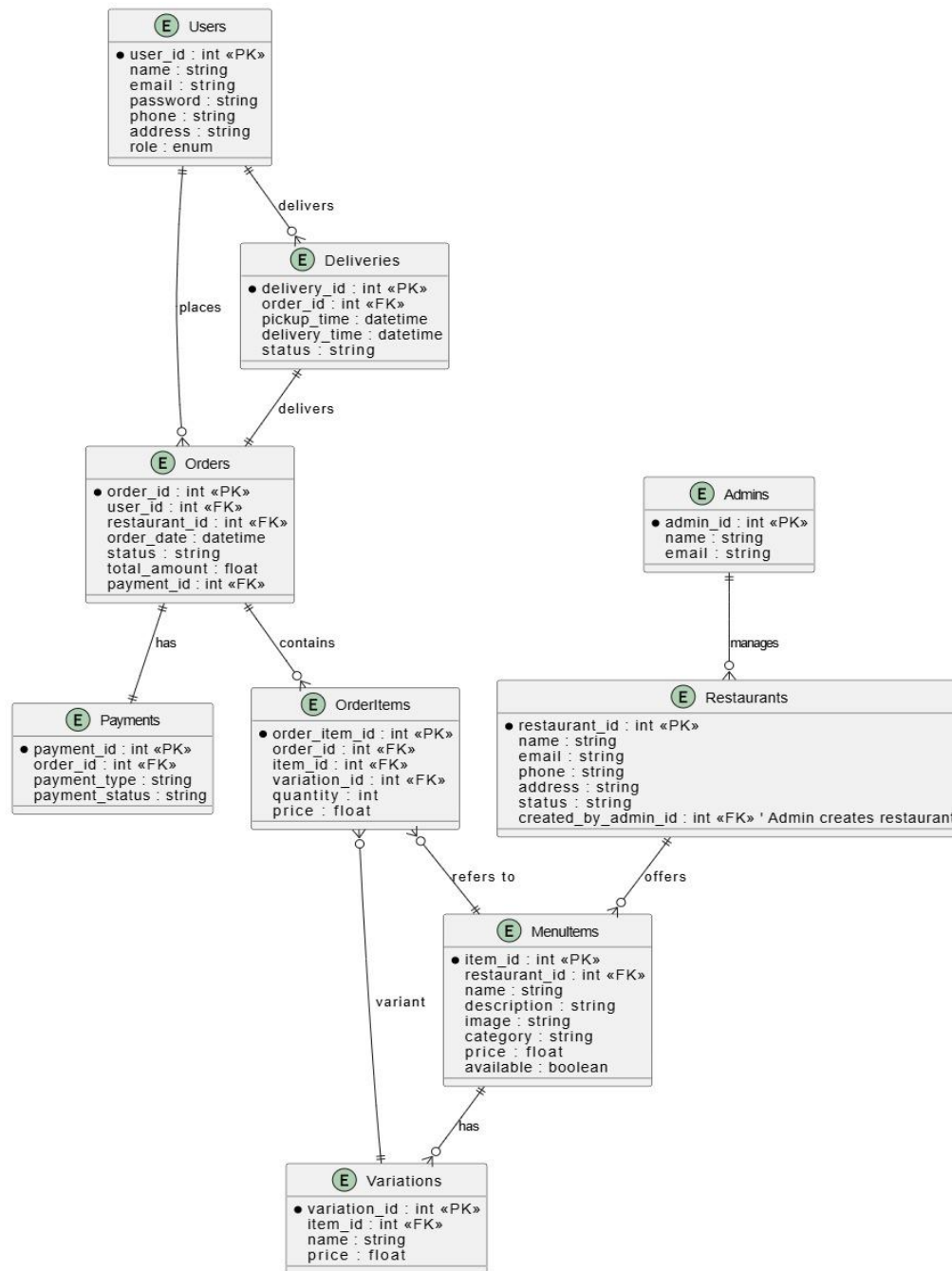


Figure 4.3 Entity Relationship Diagram

4.4 Sequence Diagram

A sequence diagram represents the flow of interactions between objects (or actors) in the system over time (see Figure 4.4, 4.5, 4.6).

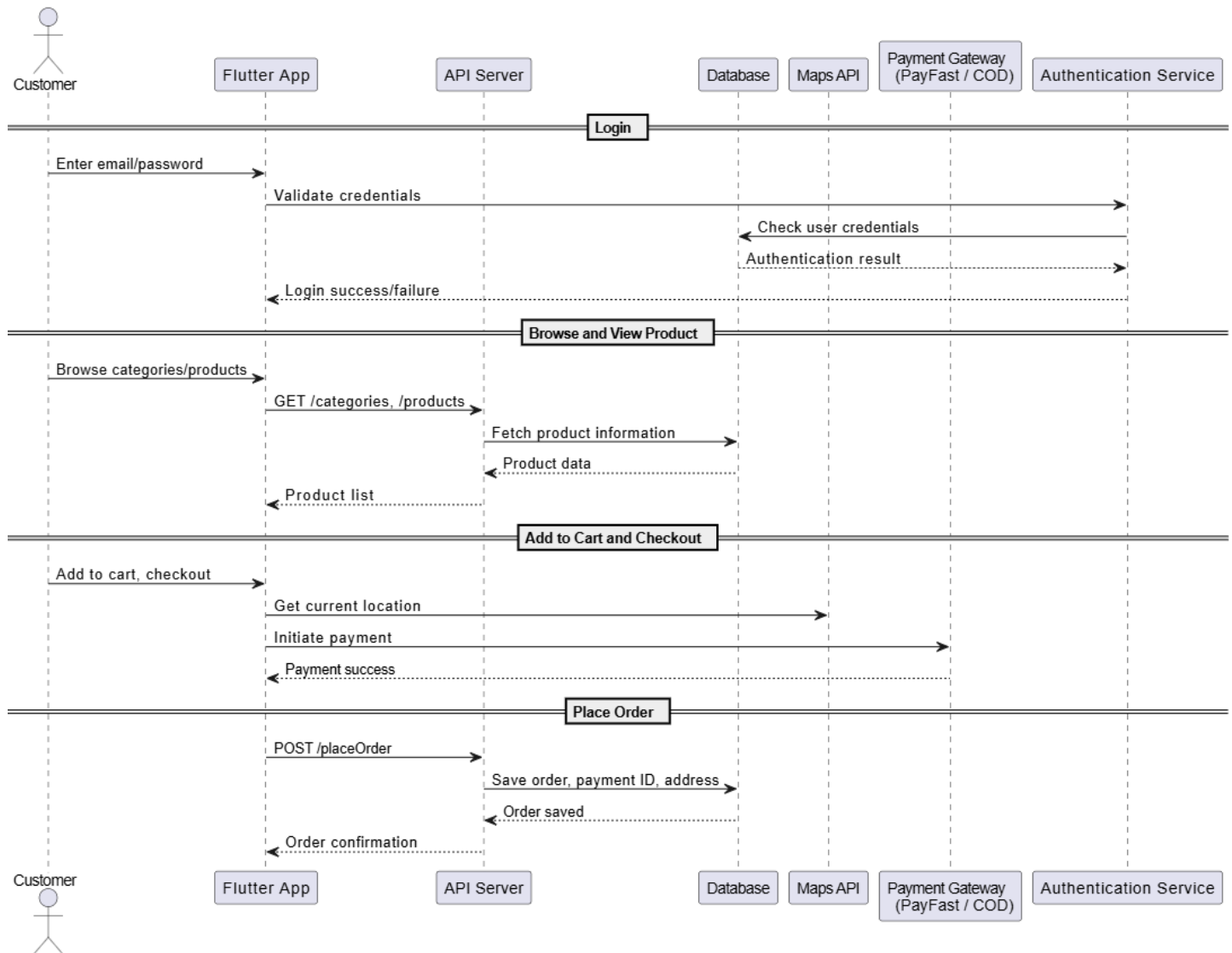


Figure 4.4 Sequence Diagram of Food app

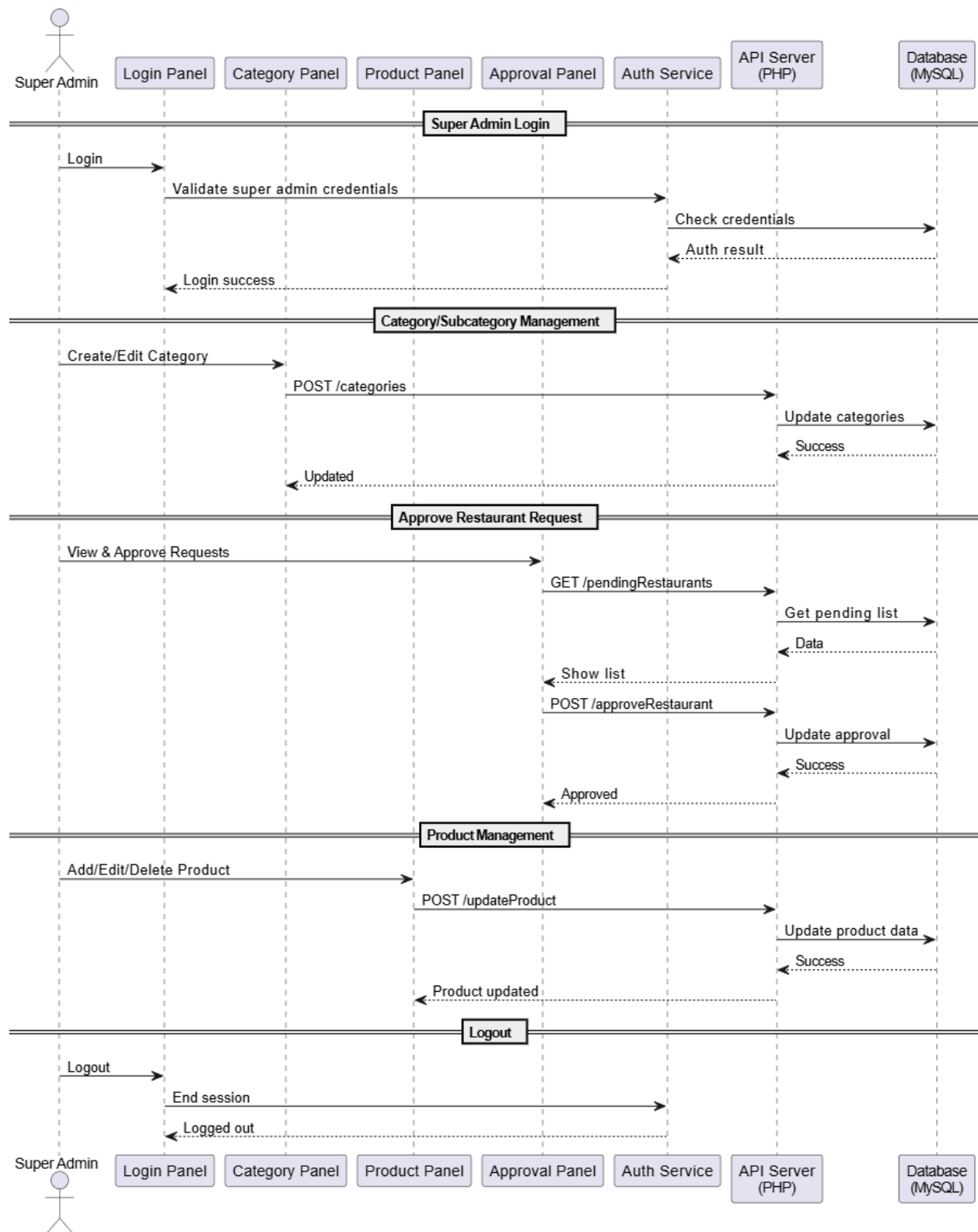


Figure 4.5 Sequence Diagram of Super Admin

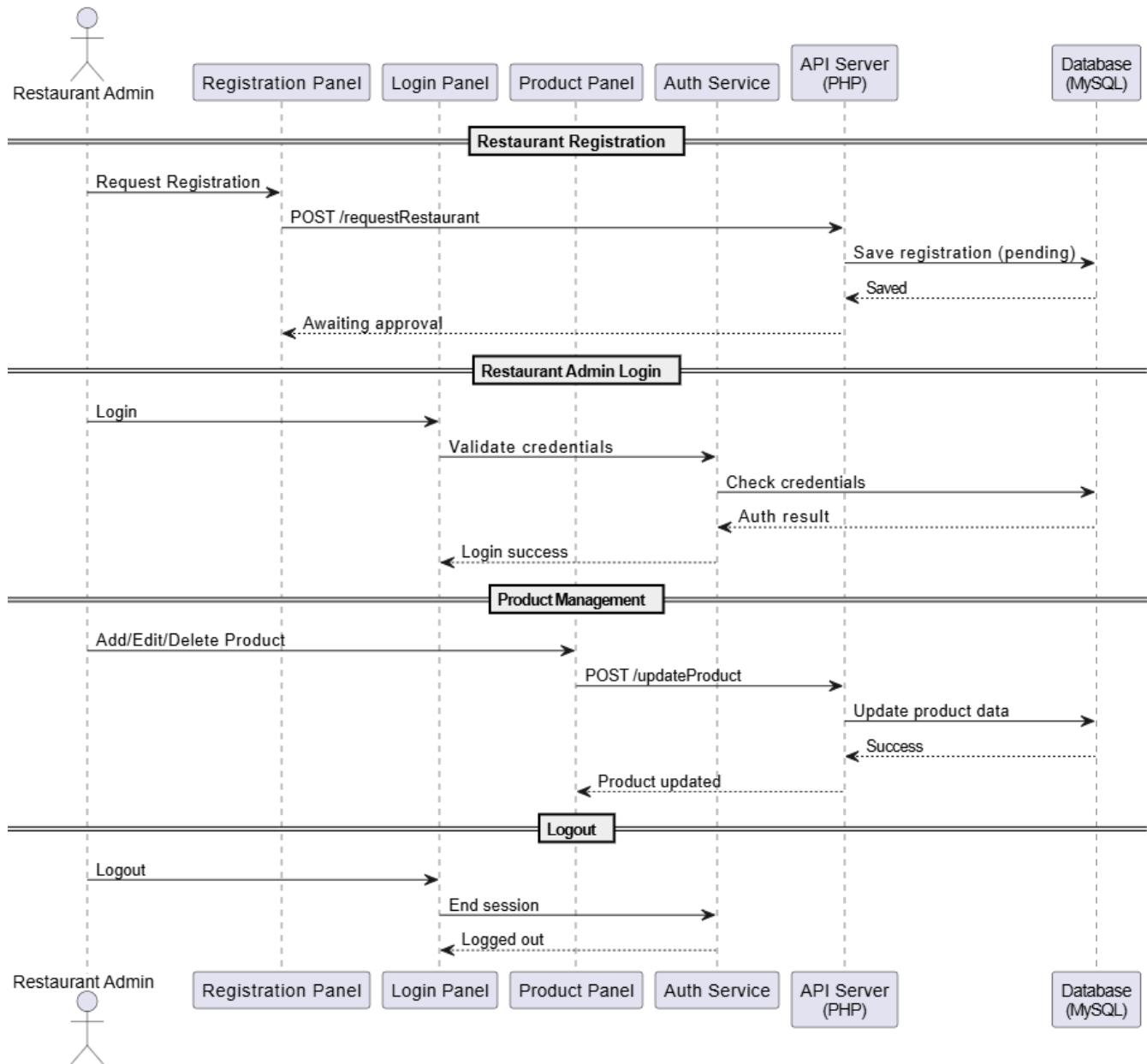


Figure 4.6 Sequence Diagram of Restaurant

Chapter No. 5

Implementation

5 Implementation of Spice Safari Food Delivery App

The implementation of the Spice Safari food delivery application involves integrating mobile and web-based front-end interfaces with backend services, managing user interactions, real-time order tracking in admin side, and seamless integration with payment gateways. This section outlines the step-by-step approach used for designing, developing, and deploying the application.

5.1 Overview

The implementation of Spice Safari focuses on enabling users to browse menus, place orders, and interact with restaurants and delivery personnel. The mobile app is developed using **Flutter (Dart)**, while the backend services, including RESTful APIs, are built using **PHP**. An **admin web portal** is created using **HTML, CSS, Bootstrap, Ajax, jQuery, JavaScript**, and **PHP** to manage orders, users, and restaurant data effectively.

5.2 Tools and Technologies

1. Programming Languages & Frameworks:

- **Frontend (App):** Flutter (Dart)
- **Backend:** RESTful APIs developed using PHP
- **Admin Portal:** HTML, CSS, Bootstrap, Ajax, jQuery, JavaScript, and PHP.

2. Databases & Services:

- **Database:** MySQL (for storing user, order, and restaurant data)
- **Authentication:** Custom PHP-based authentication or integration APIs.
- **Location Services:** Google Maps API

5.3 Modules and Their Implementation

The food delivery app is composed of functional modules that work together to provide a seamless experience for customers and admins, interacting via RESTful APIs, with PHP built for the backend and MySQL for database management.

5.3.1 User Registration and Authentication Module

The User Registration and Authentication Module enables users to sign up, log in, and manage their profiles using Flutter/Dart frontend and PHP backend. The frontend provides registration forms, while the backend handles data validation and generates authentication tokens. A MySQL database stores user information.

5.3.2 Restaurant Management Module (Admin Portal)

The admin portal manages restaurants, menu items, and order statuses using HTML, CSS, Bootstrap, and JS. The backend (PHP) handles API requests for restaurant data management, while the MySQL database stores information like restaurant names, descriptions, prices, and menu details.

5.3.3 Menu Display and Ordering Module (Customer App)

The app allows customers to browse menus, select items, add them to their cart, and place orders. It uses a Flutter/Dart frontend for display and APIs for ordering. The backend uses PHP for fetching menu items, placing orders, and retrieving order status (Pending, delivered, Cancel). A MySQL database stores customer orders.

5.3.4 Payment and Checkout Module

The app aims to streamline the order payment process by integrating payment gateways like Pay-fast and allowing users to input payment details during checkout. The backend, PHP, handles transactions, validates them.

5.3.5 Admin Dashboard and Analytics Module

The admin dashboard offers a comprehensive view of users, restaurants, orders, and analytics. Implemented using frontend (HTML, CSS, Bootstrap, JS), backend (PHP), and MySQL database, it displays statistics, manages users, and provides analytics

5.3.6 Review and Rating Module

Customers rate and review restaurants and menu items post-order, with ratings processed in PHP and stored in a MySQL database.

5.3.7 Order Tracking Module

To enable real-time tracking and status updates of customer orders, managed entirely by the admin.

5.4 Workflow

1. The app allows users to register and authenticate, manage their restaurant and menu, and place orders.
2. Users can also manage their restaurant and menu by logging into the admin portal and sending requests to the backend.
3. The app also allows users to view and order from available restaurants, with the backend processing the transactions.
4. The system stores order details, updates order status, and displays the status in the customer app.
5. Support is available through WhatsApp chats, and users can contact support representatives for assistance.
6. The app can be terminated securely, allowing for further interactions.

5.5 Testing and Deployment

Testing helps identify and fix bugs, ensure performance standards are met, and verify that all features work as expected.

1. Testing:

Unit testing is a process used to test individual app components, such as widgets and logic layers, using tools like Flutter test framework and PHPUnit for backend logic. It ensures correct authentication responses based on user IDs.

2. Deployment:

The app's production phase involves setting up environments, continuous integration/continuous deployment, and monitoring. The backend is deployed on cloud services, with a MySQL database hosted on the same server. The mobile app is deployed via Flutter/Dart, and the admin portal is deployed on a web server.

5.6 Challenges and Solutions

1. Real-time Order Tracking

Challenge: Managing real-time location sharing and order status updates. **Solution:** Implement location access only for admins and use structured API-based updates to reflect order status reliably in the app.

2. Database Performance

Challenge: Handling multiple simultaneous orders and user queries on a MySQL database. **Solution:** Optimize database queries, use indexing on key columns, and manage concurrent requests through efficient API logic.

3. Limited Customer Support Integration

Challenge: Lack of built-in messaging system for handling customer issues. **Solution:** Integrate WhatsApp support using direct chat links or APIs to enable real-time issue resolution outside the app.

5.7 Home Screen

The home screen simplifies food ordering with a search bar for specific dishes and categorized options like Fried Items and Fast Food. It features a nutrition inquiry button, recommended dishes, and a cart button for easy access to orders (see Figure 5.1).

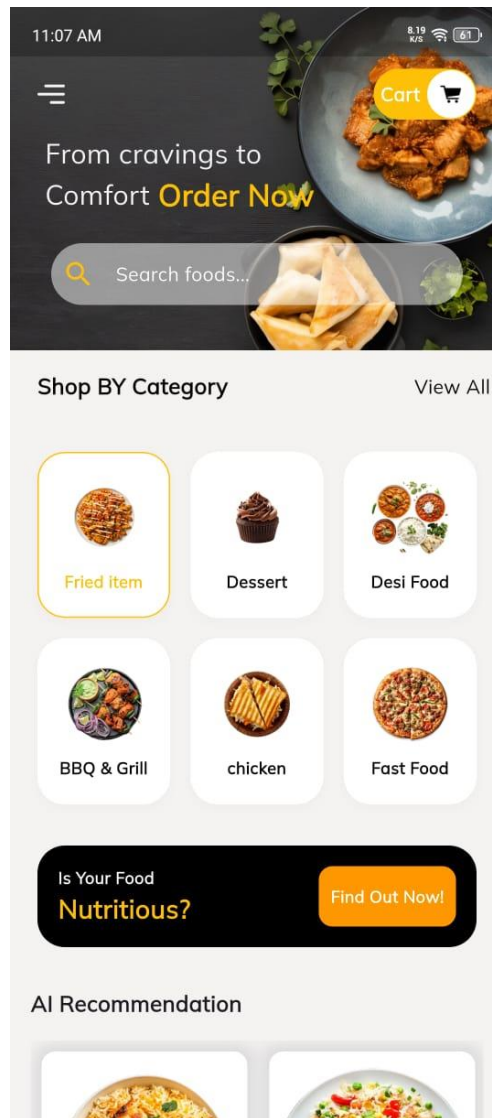


Figure 5.1 Main Screen

5.8 Categories and Subcategories

The system organizes food items into categories and subcategories to help users easily find and browse their desired dishes (see Figure 5.2).

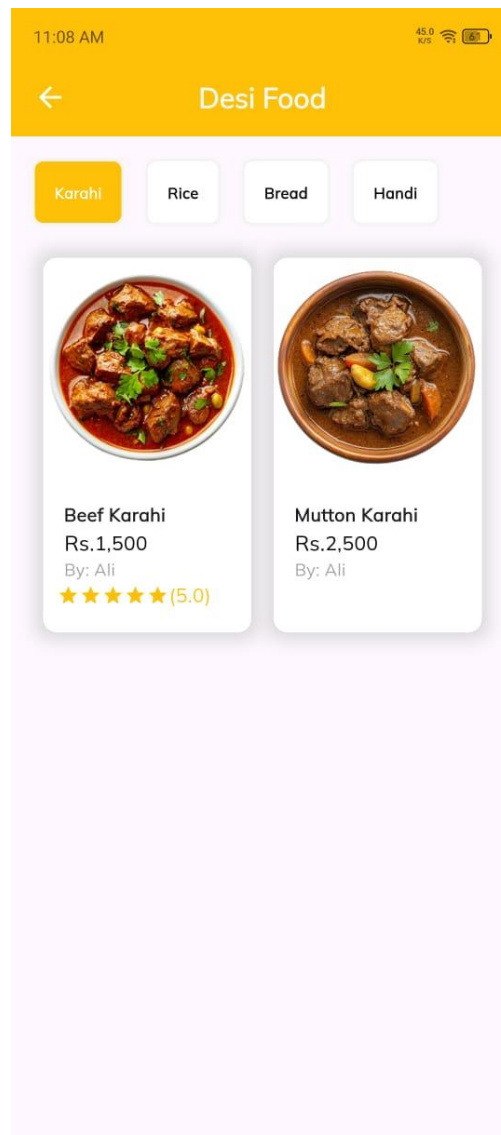


Figure 5.2 Subcategory

5.9 Cart

The cart allows users to adjust item quantities and delete unwanted items before checkout for a smooth ordering experience. (see Figure 5.3).

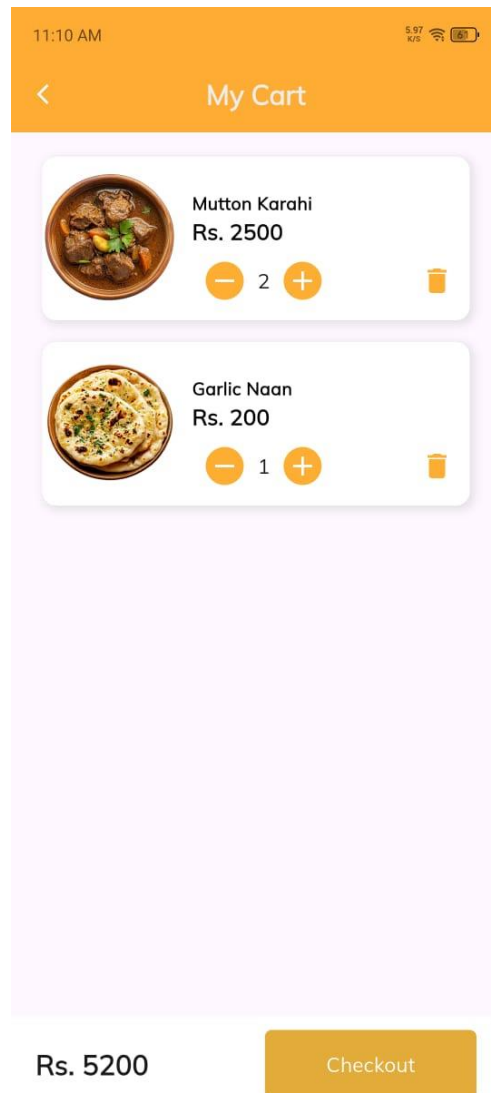
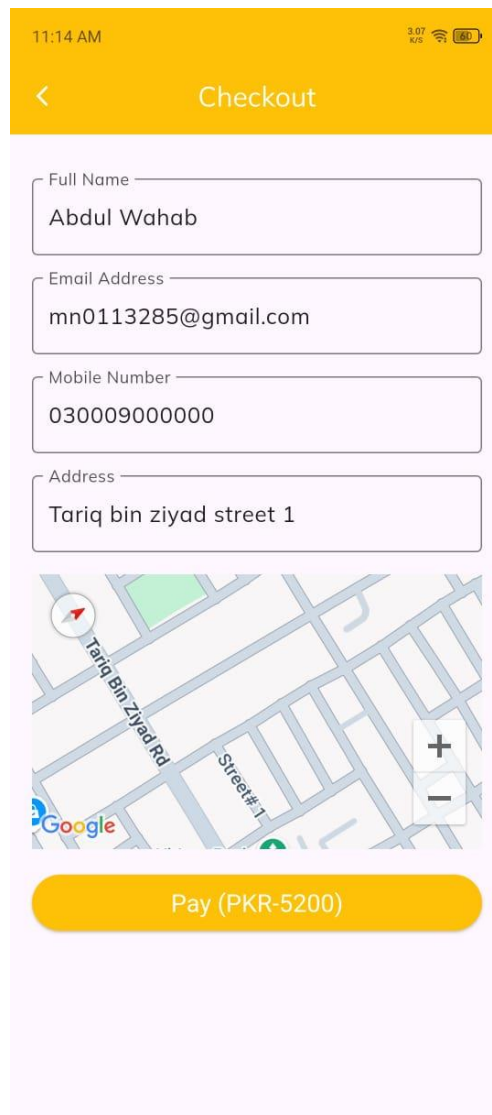


Figure 5.3 Cart

5.10 Checkout

The checkout screen collects the customer's full name, email address, mobile number, and delivery address. It also displays an interactive map to confirm the delivery location. At the bottom, a clear and prominent payment button shows the total amount to be paid, allowing users to complete their order smoothly (see Figure 5.4).



11:14 AM 3.07 4/5

< Checkout

Full Name
Abdul Wahab

Email Address
mn0113285@gmail.com

Mobile Number
030009000000

Address
Tariq bin ziyad street 1

Google

Tariq Bin Ziyad Rd Street#1

Pay (PKR-5200)

Figure 5.4 Checkout

5.11 Restaurant Admin

The Restaurant panel allows management of products to organize menu items effectively. Restaurants can be registered and managed through the system. Admins can add, update, or delete products, as well as view and view customer orders. The panel supports secure user logout for safe access control (see Figure 5.5).

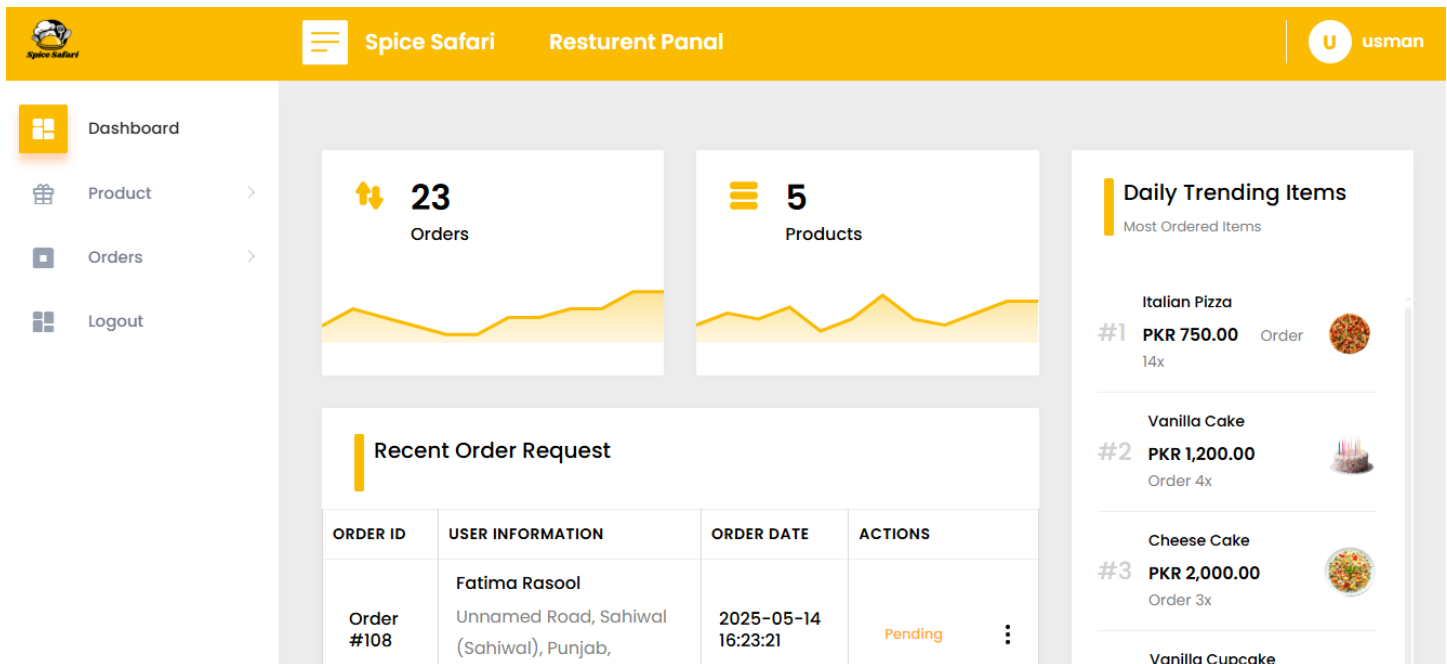


Figure 5.5 Restaurant Admin

5.12 Add Categories

The admin panel allows the super admin to create and manage categories and subcategories to organize the menu structure. Restaurant admins can then add, edit, delete, and update items including images within these predefined categories and subcategories. This setup helps maintain consistency while giving restaurants control over their menu offerings. Additionally, the panel lets admins register restaurants, manage orders, view order data, and track popular food items for better decision-making. (see Figure 5.6).

 ☰ Spice Safari Admin Panel A Admin				
Dashboard Categories Manage Categories Manage Sub Cat Product Orders Resturents Customer	Categories ID	Categories Name	Categories Image	Action
	405	Desi Food		EditDelete
	407	Fast Food		EditDelete
	408	Dessert		EditDelete
	412	Drinks		EditDelete
	413	BBQ & Grill		EditDelete
	414	Fried item		EditDelete

Figure 5.6 Add Categories Super Admin

Chapter No. 6

Testing and Evaluation

6 Testing and Evaluation

This chapter outlines the testing methodologies used to validate the functionality, reliability, and user experience of the food delivery application. The testing was applied across the Flutter mobile app, PHP-based RESTful APIs, MySQL database, and the web-based admin portal. Both manual and structured testing approaches were adopted to ensure that all modules performed as expected.

6.1 Manual Testing

Manual testing was conducted to simulate real-world user scenarios. Each module was tested separately to validate its specific function and behavior. Functional flows such as placing orders, viewing order status, admin login, assigning riders, and updating order progress were tested across various devices.

6.2 System Testing

System testing was performed after development was completed to validate the app as a whole. It ensured proper communication between the mobile app, the PHP backend, the MySQL database, and the admin portal. All features were tested end-to-end under various use cases. Any bugs or inconsistencies were reported and resolved to stabilize the final release.

6.3 Unit Testing

Unit testing focused on individual features and modules of the application.

1. For the order status update module

Tests confirmed that admins could change the order status (e.g., from “Pending” to “Delivered”) and that these updates were correctly reflected in the app via API. Each unit was tested in isolation to confirm it performed as expected, and issues were resolved promptly during development.

6.4 Functional Test Cases

To verify smooth interaction between the admin portal, customer app, backend APIs, and database, ensuring real-time updates and consistent data across all modules.

Table 6.1: Customer Order Placement Testing

No	Test Case	Attribute and Value	Expected Result	Result
1	Place order with valid details	Item: "Pizza", Payment: COD	Order stored in database	Pass
2	Place order with missing address	Address field left empty	Error message shown	Pass
3	Cancel order	Order ID: 1453, Status: "Pending"	Order marked as cancelled	Pass
4	Reorder previous item	Order ID: 1321	Item ordered again	Pass
5	Search by dish name	Search: "Burger"	List of burgers displayed	Pass
6	Search by restaurant name	Search: "Eatopiar"	Restaurant and menu shown	Pass
7	Edit customer profile	New name and phone number submitted	Profile updated successfully	Pass

6.5 Authentication and Session Management Testing

Test cases verify system login process and session behavior, ensuring valid credentials access, error messages, and session expiry for security, ensuring controlled admin panel access and user control.

Table 6.2: Admin Login and Dashboard Access

No	Test Case	Attribute and Value	Expected Result	Result
1	Valid admin credentials	Email: admin@portal.com	Access to dashboard	Pass
2	Invalid login	Wrong password	Error message displayed	Pass
3	Session timeout	Idle > 10 minutes	Redirect to login page	Pass

6.6 Integration Testing

To verify smooth interaction between the admin portal, customer app, backend APIs, and database, ensuring real-time updates and consistent data across all modules.

Table 6.3 Integration Testing

No	Test Case	Modules Involved	Expected Result	Result
1	Admin updates order status	Admin Portal to API to Database	Status change is saved in database and reflected across all systems	Pass
2	API	Customer App to	Customer app	Pass

	fetches updated order info	API to Database	displays the latest order status accurately	
3	Real-time sync of order data	Admin Portal to Database to Customer App and vice versa	Changes made in admin portal are immediately visible in the customer app	Pass

6.7 User Acceptance Testing (UAT)

User Acceptance Testing was conducted with a group of testers acting as customers and admins. They used the system in real-time scenarios to validate its usability and performance. UAT focused on whether the app was intuitive, if orders could be placed easily, if the admin could manage orders without confusion, and if WhatsApp support was accessible when needed. The feedback was positive, and minor adjustments were made to improve the admin dashboard interface and simplify rider assignment.

6.8 Summary

All major components of the food delivery system were tested through a combination of manual, unit, integration, and user acceptance testing methods. The results confirmed that the system was functional, stable, and ready for deployment. With key features operating reliably across all interfaces, the application successfully met the project's objectives and user expectations.

Chapter No. 7

Conclusion and Future Work

7 Conclusion

In conclusion, the Food Delivery App system provides an efficient, user-friendly platform that connects customers, restaurant staff, and administrators within a single, cohesive environment. Developed using Flutter for the mobile app, PHP for backend APIs, and MySQL for data storage, the system ensures smooth and responsive interactions across all components. The web-based admin portal adds robust control for order management, and real-time order status tracking (pending, cancelled, deliver). The platform is designed to streamline the ordering and delivery process, reduce manual workload, and offer clear visibility into operations. The integration of WhatsApp-based customer support also helps maintain communication without complicating the core system. By centralizing order management, the application simplifies logistics while maintaining flexibility and scalability. This system not only improves food delivery operations for users and businesses alike but also serves as a solid foundation for expansion and technological enhancement. With its modular architecture and scalable infrastructure, it can be adapted to support larger geographic areas, more users, and additional features as needed.

8 Future Work

To further enhance the functionality and user experience of the food delivery system, several future developments and improvements are planned:

8.1 Automated Rider Assignment

System Develop and implement an intelligent algorithm to automatically assign available riders to orders based on location, availability, and delivery history. This will reduce admin workload and minimize delivery delays.

8.2 Loyalty and Reward System

Introduce a points-based system where customers earn rewards for repeat orders, referrals, or reviews. Enhancing user retention and engagement through gamified incentives.

8.3 Performance Analytics Dashboard

Add analytics and visual reports in the admin portal to monitor daily sales, delivery times, order status distribution, and rider performance. This data-driven approach will help businesses make informed decisions.

8.4 Enhanced Security and Data Privacy

Strengthen data encryption and implement role-based access controls to safeguard user data and administrative operations. Ensures compliance with data protection regulations and enhances user trust.

By implementing these future enhancements, the food delivery system will continue to evolve and stay competitive in a growing and dynamic market. With a focus on automation, scalability, and user satisfaction, it is well-positioned to become a comprehensive solution for modern food service logistics.

9 References

- [1] Emerald insight, "The adoption of food delivery apps," 2023.
- [2] P. (n.d.), "How Online Ordering Systems Work," 26 Jan 2024.
- [3] M. Ritha, "Online Food Delivery Apps in the Modern Era: A Contemporary Perspective," January 2025.
- [4] Saranya, "Customer Satisfaction with Online Food Delivery Services," 2024.